Lab 01 · Stacks, change sets y drift con CloudFormation

Duración estimada: 50 minutos **Sesión:** 1 **Entorno:** GitHub Codespaces con un emulador local de AWS · sin cuenta de AWS

Objetivo

Desplegar un stack de CloudFormation parametrizado, modificarlo de forma controlada mediante un change set (incluido un cambio que en AWS obliga a reemplazar un recurso) y detectar y corregir un cambio manual hecho fuera de la plantilla.

Cómo funciona el entorno

Trabajas en un **Codespace**: una máquina Linux en la nube de GitHub que se abre en el navegador, con un editor y una terminal. Dentro corre un **emulador de AWS** (Floci) en `http://localhost:4566`. La CLI de AWS ya está configurada para hablar con él, así que los comandos `aws ...` son **exactamente los mismos** que usarías contra AWS real; lo único que cambia es la variable de entorno `AWS_ENDPOINT_URL`, que apunta al emulador. Nada de lo que hagas cuesta dinero ni toca ninguna cuenta real.

> **Lo que el emulador no hace igual que AWS.** Un emulador reproduce las API, no todo el comportamiento. En esta práctica hay tres diferencias, señaladas donde aparecen: la validación de plantillas es mínima (por eso usamos `cfn-lint`), el change set puede no indicar si un cambio provoca **reemplazo**, y la detección automática de drift puede no estar disponible. En cada caso se explica qué verías en AWS real.

Requisitos

- Una cuenta gratuita de GitHub y un navegador. No hace falta instalar nada.

Punto de partida

1. Abre el repositorio del curso en GitHub, pulsa **Code → Codespaces → Create codespace on master** y espera a que cargue (la primera vez, 2-4 minutos).
2. En la terminal (parte inferior; si no aparece, menú ☰ **→ Terminal → New Terminal**), comprueba que el emulador responde:

```
aws sts get-caller-identity
```

Debe devolver una cuenta ficticia. Si da error, ejecuta `bash .devcontainer/arrancar-emulador.sh` y repite.

Paso 1 · Preparar la plantilla

```
export STACK=lab01
mkdir -p ~/lab01 && cd ~/lab01

cat > plantilla.yaml <<'EOF2'
AWSTemplateFormatVersion: "2010-09-09"
Description: Lab 01 - bucket de datos y tema de avisos

Parameters:
  Entorno:
    Type: String
    AllowedValues: [dev, pre, pro]
    Default: dev
  Sufijo:
    Type: String
    AllowedPattern: "[a-z0-9-]+"
    Default: alumno
```

```
Resources:
  BucketDatos:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: { "Fn::Sub": "lab01-datos-${Sufijo}-${AWS::AccountId}" }
      VersioningConfiguration:
        Status: Enabled
      BucketEncryption:
        ServerSideEncryptionConfiguration:
          - ServerSideEncryptionByDefault:
              SSEAlgorithm: AES256
      Tags:
        - Key: entorno
          Value: { Ref: Entorno }
        - Key: curso
          Value: devops-aws

  TemaAvisos:
    Type: AWS::SNS::Topic
    Properties:
      TopicName: { "Fn::Sub": "lab01-avisos-${Sufijo}" }

Outputs:
  NombreBucket:
    Value: { Ref: BucketDatos }
  ArnTema:
    Value: { Ref: TemaAvisos }
EOF2
```

> Escribimos las funciones intrínsecas en su **forma larga** (`{ "Fn::Sub": ... }`, `{ Ref: ... }`). En AWS real es equivalente a la forma corta `!Sub` y `!Ref`; la forma larga es la que admiten todas las herramientas y emuladores sin excepción.

Paso 2 · Validar y desplegar

Valida la plantilla con `cfn-lint`, que ya está instalado en el Codespace:

```
cfn-lint plantilla.yaml && echo "Plantilla correcta"
```

> `aws cloudformation validate-template` también existe, pero en el emulador apenas comprueba nada. `cfn-lint` es más estricto incluso que AWS real: revisa tipos de recurso, propiedades y valores permitidos. Úsalo siempre, también en tus proyectos.

Despliega el stack:

```
aws cloudformation deploy \
  --stack-name $STACK \
  --template-file plantilla.yaml \
  --parameter-overrides Entorno=dev \
  --tags curso=devops-aws
```

Consulta el estado, los recursos y las salidas:

```
aws cloudformation describe-stacks --stack-name $STACK \
  --query "Stacks[0].[StackStatus]" --output text

aws cloudformation list-stack-resources --stack-name $STACK \
  --query "StackResourceSummaries[].[LogicalResourceId,ResourceType,ResourceStatus]" --output table

aws cloudformation describe-stacks --stack-name $STACK \
  --query "Stacks[0].Outputs" --output table
```

```
export BUCKET=$(aws cloudformation describe-stacks --stack-name $STACK \
  --query "Stacks[0].Outputs[?OutputKey=='NombreBucket'].OutputValue" --output text)
echo $BUCKET
```

> **Punto de control 1.** El stack está en `CREATE_COMPLETE`, tiene dos recursos y las salidas muestran el nombre del bucket y el ARN del tema.

Paso 3 · Change set con un cambio y un reemplazo

Introducimos dos cambios:

- Una regla de ciclo de vida en el bucket (cambio **sin** reemplazo).
- Un cambio en `TopicName` del tema SNS. En AWS esta propiedad **no se puede modificar** en un recurso existente, así que obliga a **reemplazarlo**: crear uno nuevo y borrar el antiguo.

```
cp plantilla.yaml plantilla_v2.yaml

python3 - <<'EOF2'
p = "plantilla_v2.yaml"
s = open(p).read()
s = s.replace("""      Tags:
        - Key: entorno""", """      LifecycleConfiguration:
        Rules:
          - Id: expirar-versiones-antiguas
            Status: Enabled
            NoncurrentVersionExpiration:
              NoncurrentDays: 30
      Tags:
        - Key: entorno""")
s = s.replace('"lab01-avisos-${Sufijo}"', '"lab01-alertas-${Sufijo}"')
open(p, "w").write(s)
EOF2

diff plantilla.yaml plantilla_v2.yaml
cfn-lint plantilla_v2.yaml && echo "v2 correcta"
```

Crea el change set **sin ejecutarlo**:

```
aws cloudformation create-change-set \
  --stack-name $STACK \
  --change-set-name cambio-v2 \
  --template-body file://plantilla_v2.yaml \
  --parameters ParameterKey=Entorno,UsePreviousValue=true ParameterKey=Sufijo,UsePreviousValue=true

sleep 5
aws cloudformation describe-change-set \
  --stack-name $STACK --change-set-name cambio-v2 \
  --query "Changes[].ResourceChange.[LogicalResourceId,Action,Replacement]" \
  --output table
```

> **Punto de control 2.** Ves `BucketDatos` y `TemaAvisos` con acción `Modify`. **En AWS real**, la columna `Replacement` mostraría `False` para el bucket y `True` para el tema, y la consola señalaría `TopicName` como la propiedad que provoca el reemplazo. Si el emulador muestra `None` o no rellena esa columna, es una de sus limitaciones: la decisión de reemplazar la toma AWS según la documentación de cada propiedad (*Update requires: Replacement*), y consultarla antes de cambiar una propiedad es parte del trabajo.

Pregunta: si en lugar de un tema SNS fuese una base de datos con datos de clientes, ¿ejecutarías este change set? ¿Qué harías antes?

Ejecuta el change set y comprueba el resultado:

```
aws cloudformation execute-change-set --stack-name $STACK --change-set-name cambio-v2
sleep 5
aws cloudformation describe-stacks --stack-name $STACK --query "Stacks[0].[StackStatus,Outputs]" --output
aws sns list-topics --output table
```

Fíjate en el ARN del tema: ahora termina en `lab01-alertas-alumno`. Es un recurso nuevo.

Paso 4 · Provocar y detectar drift

Simula que alguien cambia a mano una etiqueta del bucket, fuera de CloudFormation:

```
aws s3api get-bucket-tagging --bucket $BUCKET --output table

aws s3api put-bucket-tagging --bucket $BUCKET \
  --tagging 'TagSet=[{Key=entorno,Value=manual},{Key=curso,Value=devops-aws}]'

aws s3api get-bucket-tagging --bucket $BUCKET --output table
```

Intenta la detección automática de drift:

```
aws cloudformation detect-stack-drift --stack-name $STACK
```

Si el comando devuelve un identificador, sigue con:

```
sleep 5
aws cloudformation describe-stack-resource-drifts --stack-name $STACK \
  --query "StackResourceDrifts[].[LogicalResourceId,StackResourceDriftStatus]" --output table
```

Si devuelve un error de operación no implementada, haz la detección **a mano**, que es exactamente lo que hace CloudFormation por dentro: comparar el valor declarado con el real.

```
DECLARADO=$(aws cloudformation describe-stacks --stack-name $STACK \
  --query "Stacks[0].Parameters[?ParameterKey=='Entorno'].ParameterValue" --output text)
REAL=$(aws s3api get-bucket-tagging --bucket $BUCKET \
  --query "TagSet[?Key=='entorno'].Value" --output text)
echo "Declarado en la plantilla: $DECLARADO · Valor real: $REAL"
[ "$DECLARADO" = "$REAL" ] && echo "IN_SYNC" || echo "DRIFTED"
```

> **Punto de control 3.** El resultado es `DRIFTED`: la plantilla dice `dev` y el recurso tiene `manual`. **En AWS real**, `detect-stack-drift` marcaría el stack como `DRIFTED` y `describe-stack-resource-drifts` mostraría la propiedad `Tags` con el valor esperado y el real, para todos los recursos del stack a la vez.

Paso 5 · Corregir el drift desde la plantilla

La forma correcta de corregir el drift es que CloudFormation vuelva a imponer el estado declarado. Cambia el valor declarado del parámetro y despliega:

```
aws cloudformation deploy \
  --stack-name $STACK \
  --template-file plantilla_v2.yaml \
  --parameter-overrides Entorno=pre \
  --tags curso=devops-aws

aws s3api get-bucket-tagging --bucket $BUCKET --output table
```

Repite la comprobación del paso 4.

> **Observación.** Si redespliegas sin cambiar nada, CloudFormation dice que no hay cambios y **no** corrige el drift:

> solo aplica lo que cambia en la plantilla o en los parámetros. Esta práctica muestra por qué los cambios manuales son peligrosos: el estado real y el declarado se separan sin que nadie lo sepa hasta que se detecta.

Reto opcional

Activa la protección contra borrado del stack e intenta eliminarlo:

```
        aws cloudformation update-termination-protection --stack-name $STACK --enable-termination-protection
        aws cloudformation delete-stack --stack-name $STACK
```

Observa la respuesta. Después desactívala para poder limpiar. Si el emulador no la respeta, en AWS real el borrado devolvería un error de validación.

Preguntas de reflexión

1. ¿Por qué el nombre del bucket incluye el sufijo y el identificador de la cuenta?
2. ¿Qué dos mecanismos de CloudFormation te protegen de borrar por accidente un recurso con datos?
3. ¿Qué ventajas e inconvenientes tiene probar infraestructura en un emulador local frente a una cuenta de AWS de pruebas?

Limpieza

```
        aws cloudformation update-termination-protection --stack-name $STACK --no-enable-termination-protection
2>/dev/null
        aws s3 rm s3://$BUCKET --recursive
        aws cloudformation delete-stack --stack-name $STACK
```

Al cerrar el Codespace todo desaparece. Para no consumir horas gratuitas, deténlo al terminar: en GitHub, **Code → Codespaces → ⋯ → Stop codespace**.